



Building Java Programs
A BACK TO BASICS APPROACH 5e

Fundamentals of Computer Science

II

05 - Generic - LinkedList

Instructor: Varik Hoang



Objectives

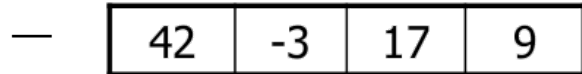
- Linked Data Structure
- What is Node?
- Construct a List
 - ListNode class
 - Data Access/Modification
- Reference & Object
- Re-assigning Reference
- List Traversal
- Inner Class



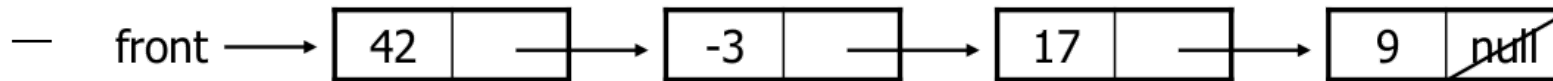
Linked Data Structure

All of the collections we will use and implement in this course:

- An array of **all elements** (ArrayList, Stack, HashSet, HashMap)

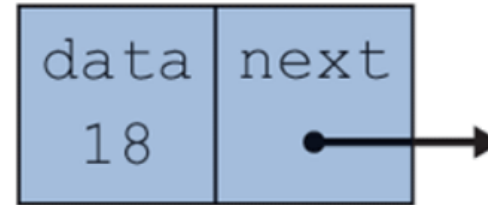


- A set of linked objects, **each storing one element**, and one or more reference(s) to other element(s). (**LinkedList**, TreeSet, TreeMap)



Node

- A single element of a structure such as a linked list
- Each node contains:
 - one data *value*
 - one *reference* to the *next* node



```
class ListNode
{
    int data;
    ListNode next;
}
...
ListNode node = new ListNode();
node.data = 18;
node.next = null;
```



Constructing a List

- Construct a series of nodes that store the sequence of values [3, 7, 12]

```
ListNode list;
```

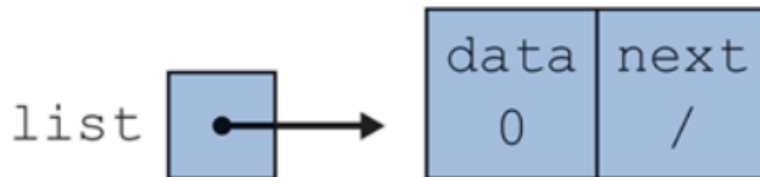
list 



Constructing a List (cont.)

- Create an actual node by calling **new** keyword

```
ListNode list;  
list = new ListNode();
```

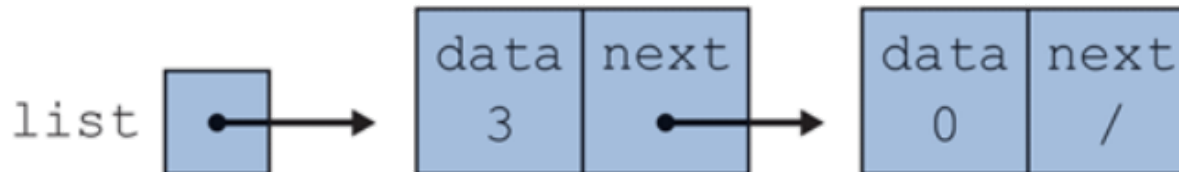




Constructing a List (cont.)

- Assign the value **3** and next node reference by...
 - Assigning values to **.data** and **.next**

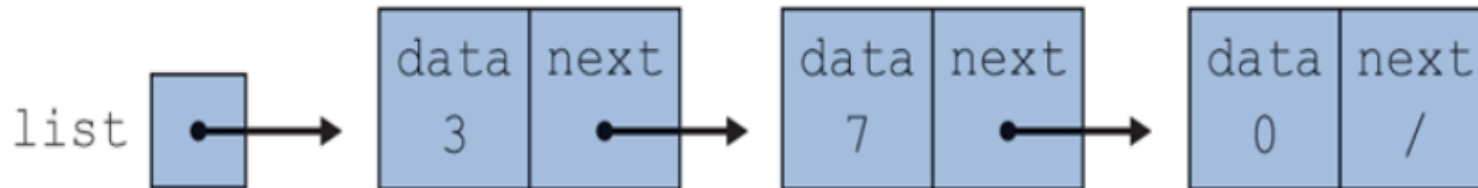
```
ListNode list;  
list = new ListNode();  
list.data = 3;  
list.next = new ListNode();
```



Constructing a List (cont.)

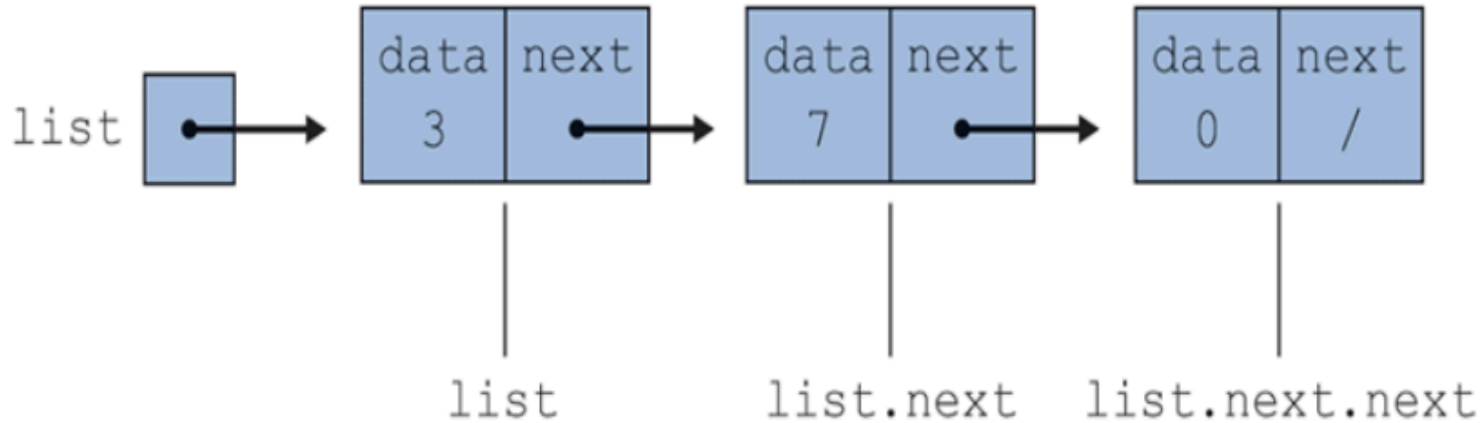
- Assign the value **7** and next node reference by...
 - Assigning values to **.data** and **.next**

```
ListNode list;  
list = new ListNode();  
list.data = 3;  
list.next = new ListNode();  
list.next.data = 7;  
list.next.next = new ListNode();
```



Constructing a List (cont.)

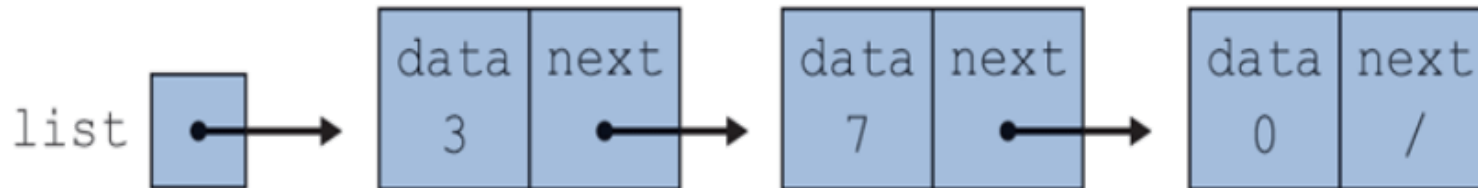
- Pay close attention to the difference between **list**, **list.next** and **list.next.next**



Constructing a List (cont.)

- Assign the value **12** and next node reference by...
- Assigning values to **.data** and **.next**

```
ListNode list;  
list = new ListNode();  
list.data = 3;  
list.next = new ListNode();  
list.next.data = 7;  
list.next.next = new ListNode();  
list.next.next.data = 12;  
list.next.next.next = null;
```

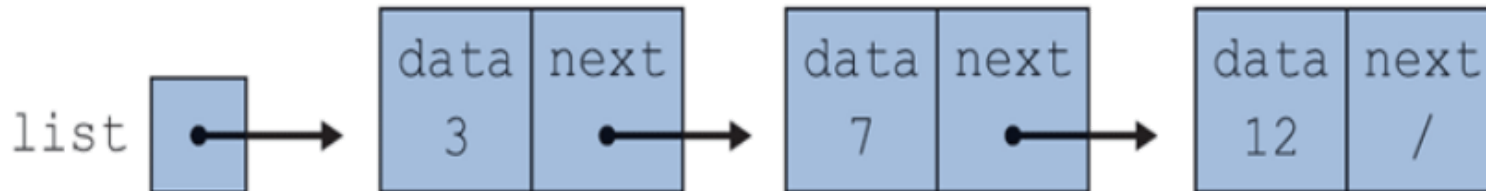


Constructing a List (cont.)

- Print all values in the linked list (only have reference to the myList)

```
ListNode list;  
list = new ListNode();  
list.data = 3;  
list.next = new ListNode();  
list.next.data = 7;  
list.next.next = new ListNode();  
list.next.next.data = 12;  
list.next.next.next = null;  
System.out.println(list.data  
    + " " + list.next.data  
    + " " + list.next.next.data);
```

3 7 12





ListNode with Constructors

```
class ListNode
{
    int data;
    ListNode next;

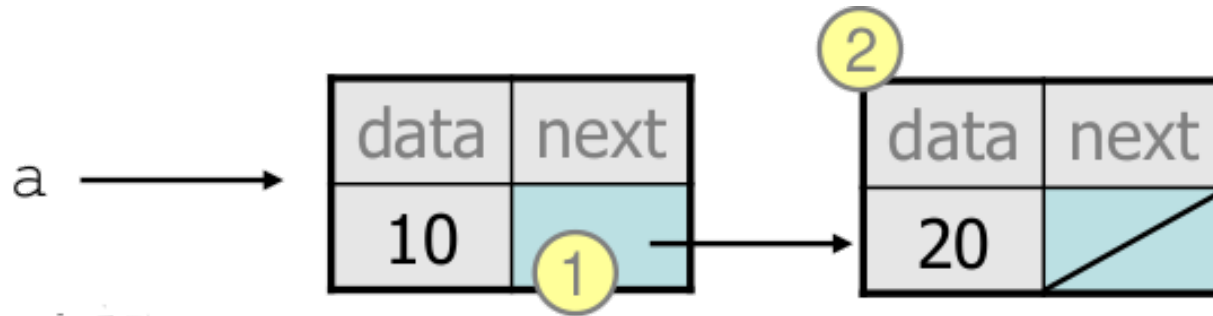
    public ListNode(int data) {
        this.data = data;
        this.next = null;
    }

    public ListNode(int data, ListNode next) {
        this.data = data;
        this.next = next;
    }
}

...
ListNode node = new ListNode(18);
node.next = new ListNode(20);
```

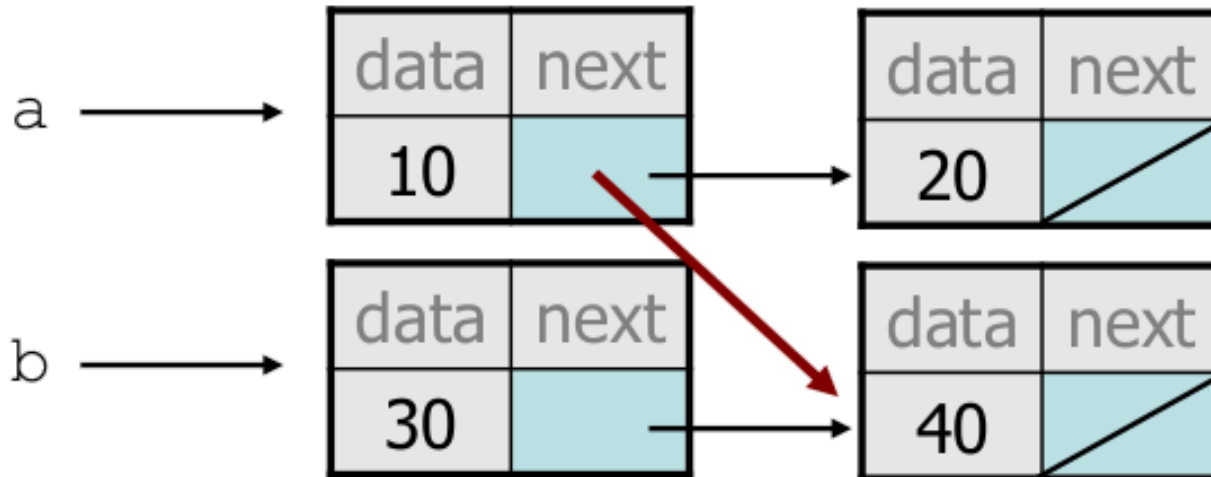
References vs. Objects

- `variable = value;`
 - a variable (left side of `=`) is an arrow (the base of an arrow)
 - a value (right side of `=`) is an object (a box what an arrow points at)
- For example:
 - `a.next = value;` // means to adjust where point to
 - `variable = a.next;` // means to make variable point at



Reassigning References

- When you say:
 - `a.next = b.next;`
- You are saying:
 - "Make the variable `a.next` refer to the same value as `b.next`." OR
 - "Make `a.next` point to the same place that `b.next` points."

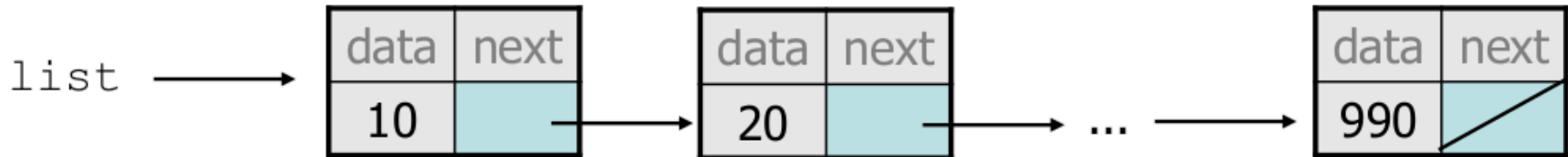




List Traversal

- Suppose we have a long chain of list nodes:
- Algorithm Pseudocode

```
while (there are more nodes to print)
    print the current node's data
    go to the next node
```



- How do we walk through the nodes of the list?

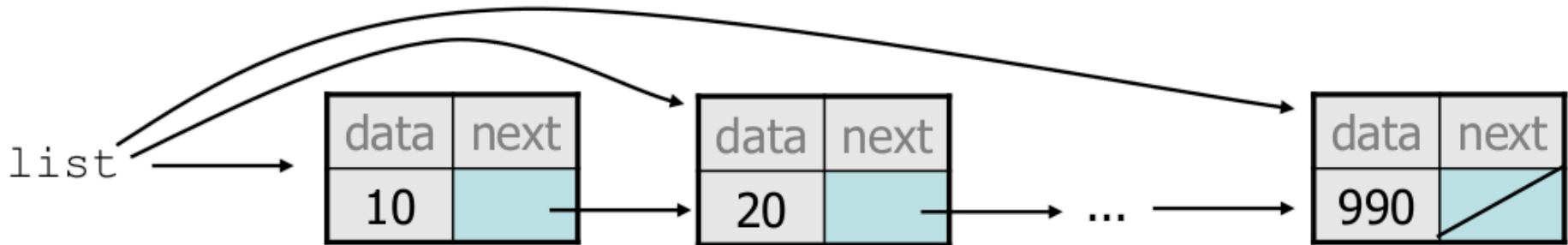
```
list = list.next; // is this a good idea?
```



List Traversal (cont.)

- One (bad) way to print every value in the list:

```
while (list != null)
{
    System.out.println(list.data);
    list = list.next; // move to the next node
}
```



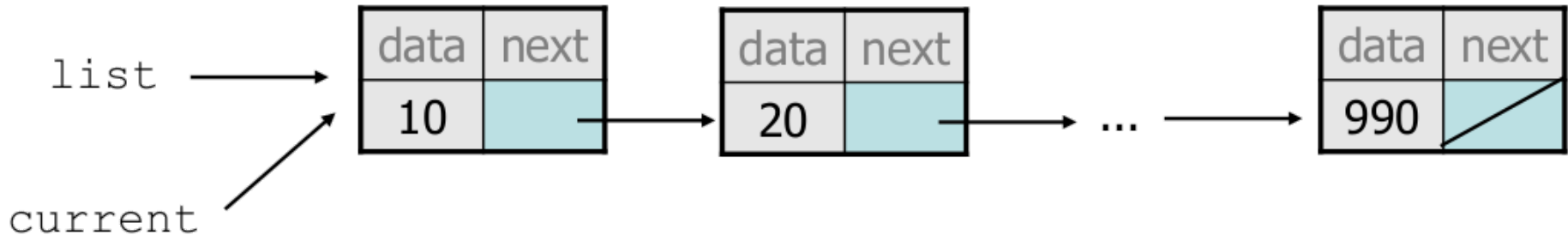
- What's wrong with this approach?



List Traversal (cont.) - A Current Reference

- Don't change list. Make another variable, and change that.

```
ListNode current = list;  
while (current != null)  
{  
    System.out.println(current.data);  
    current = current.next; // move to next node  
}
```



- A **ListNode** variable is **NOT** a **ListNode** object
- What happens after we changed the code above?



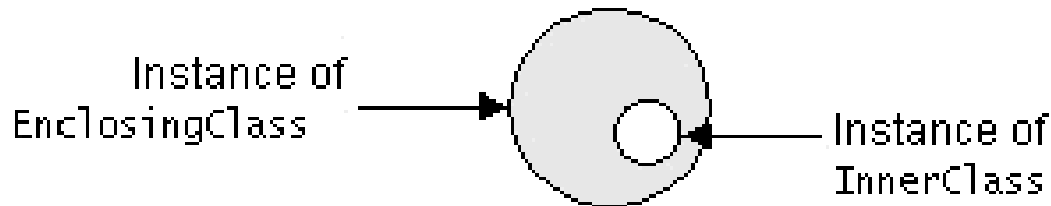
Linked List & Array List Traversal

```
ListNode current = list;  
while (current != null)  
{  
    System.out.println(current.data);  
    current = current.next; // move to next node  
}
```

```
int index = 0;  
while (index < array.length)  
{  
    System.out.println(array[index]);  
    index++;  
}
```

Inner Class

- A class **defined inside** of another class.
 - Can be created as **static** or **non-static**
 - We will focus on standard non-static ("**nested**") inner classes
- Usefulness:
 - Inner classes **are hidden** from other classes (encapsulated)
 - Inner objects can **access/modify the fields** of the outer object





Inner Class - Static & Non-static

```
class A
{
    class B // non-static inner class
    {
        // static int x; not allowed here
    }

    static class C // static inner class
    {
        static int x; // allowed here
    }
}
```

```
class Test
{
    public static void main(String[] args)
    {
        A a = new A();

        // requires enclosing instance
        A.B obj1 = a.new B();

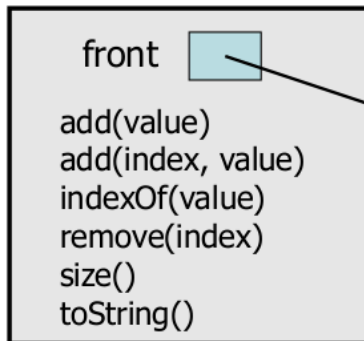
        // no need for reference of object to the outer class
        A.C obj2 = new A.C();
    }
}
```

- There are two differences between static inner and non static inner classes.
 - 1 **Non static inner** class cannot have static fields and methods but **static inner** class can
 - 2 The instance of **non static inner** class is created with the reference of object of outer class but the instance of **static inner** class is created without the reference

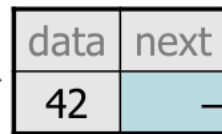
Inner Class - LinkedList & ListNode

```
public class LinkedList
{
    ListNode front;
    ...
    public class ListNode
    {
        int data;
        ListNode next;
        ...
    }
}
```

LinkedList

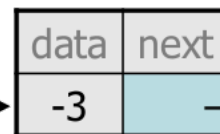


ListNode



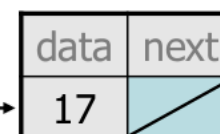
element 0

ListNode



element 1

ListNode



element 2

Questions & Answer





Thank You!

A black and silver fountain pen is positioned diagonally below the end of the 'Thank You!' text. The pen has a black barrel with silver-colored accents and a silver-colored nib. It is resting on a white surface, creating a soft reflection.